PROJETO 3 • ATIVIDADE PRÁTICA EM SALA

MISSÃO ORBITAL: SIMULADOR DE TREINAMENTO ESPACIAL

Um programa em C para validar pontuações, controlar etapas e classificar o desempenho de cadetes

MODALIDADE	REALIZAÇÃO	FORMATO
Individual ou grupos de até 3 alunos	Desenvolvimento durante a aula prática	Código-fonte em C e registro da execução

1. Contexto da missão

A Agência Orbital está selecionando equipes para uma missão de exploração. Cada cadete realizará três etapas de treinamento. O sistema deverá receber as pontuações, impedir valores inválidos, calcular o desempenho e informar a classificação final. Ao término, o operador poderá iniciar o treinamento de outro cadete ou encerrar o programa.

2. Objetivo do projeto

Desenvolver, testar e explicar um programa em linguagem C que combine estruturas condicionais e laços de repetição em uma situação completa, curta e adequada ao tempo de aula.

3. Conteúdos mobilizados

Conteúdo	Aplicação no programa
`Condicionais`	Classificar o desempenho e reconhecer a pontuação máxima.
`for`	Controlar as três etapas do treinamento.
`while`	Repetir a leitura enquanto a pontuação estiver fora do intervalo de 0 a 100.
`do-while`	Permitir o treinamento de outro cadete antes de encerrar o sistema.
`Contador e acumulador`	Identificar a etapa atual e calcular a pontuação total.

MISSÃO DO GRUPO

Tratar o problema como uma solução computacional, elaborando o algoritmo e o respetivo programa. Ao final, testar casos diferentes e conseguir explicar cada estrutura utilizada.

4. Dados do programa

Variável sugerida	Finalidade	Tipo
`codigo_cadete`	Identificação numérica do participante.	`int`
`etapa`	Controla as três etapas do treinamento.	`int`
`pontuacao`	Pontuação digitada em cada etapa, de 0 a 100.	`int`
`pontuação_total`	Acumula as três pontuações.	`int`
`media`	Armazena a média das etapas.	`float`
`continuar`	Indica se outro cadete será avaliado.	`int`

5. Funcionamento obrigatório

Etapa	O programa deverá	Estrutura
1	Ler o código numérico do cadete e zerar o total.	`Entrada e atribuição`
2	Executar exatamente três etapas de treinamento.	`for`
3	Aceitar somente pontuações entre 0 e 100. Valores inválidos devem ser solicitados novamente.	`while`
4	Somar as pontuações e calcular a média real.	`Acumulador`
5	Classificar o desempenho conforme as regras abaixo.	`if/else`
6	Perguntar se o operador deseja avaliar outro cadete.	`do … while`

6. Classificação do treinamento

Média	Resultado	Mensagem sugerida
85,0 ou mais	**Comandante da missão**	Treinamento concluído com excelência.
70,0 a 84,99	**Piloto aprovado**	Cadete autorizado para a missão.
50,0 a 69,99	**Cadete em recuperação**	Novo treinamento recomendado.
Abaixo de 50,0	**Treinamento reiniciado**	Cadete ainda não autorizado.

Condição adicional

Se o total for igual a 300 pontos, exiba também: PONTUAÇÃO MÁXIMA! Essa mensagem deve ser controlada por uma condicional simples.

7. Caso de teste

```
===== MISSÃO ORBITAL =====
Código do cadete: 17

Pontuação da etapa 1: 85
Pontuação da etapa 2: 70
Pontuação da etapa 3: 95

---------- RESULTADO ----------
Cadete: 17
Pontuação total: 250 pontos
Média: 83.33
Classificação: PILOTO APROVADO
------------------------------

Avaliar outro cadete? 1-Sim | 0-Não: 0
Sistema encerrado.
```

8. Testes mínimos

Teste	Pontuações	Média	Resultado esperado
1	85, 70 e 95	83,33	Piloto aprovado
2	90, 85 e 95	90,00	Comandante da missão
3	40, 50 e 55	48,33	Treinamento reiniciado
Validação	120, depois 80	-	Recusar 120 e aceitar 80

9. Etapas de trabalho em aula

Momento	Atividade	Referência
Planejamento	Identificar entradas, processamento, condições e repetições.	10 a 15 min
Construção	Criar o programa por partes e compilar após cada bloco.	40 a 50 min
Testes	Executar os quatro testes mínimos e corrigir os resultados.	15 a 20 min
Entrega	Organizar o arquivo, registrar a execução e explicar a solução.	10 a 15 min

10. O que deverá ser entregue

- arquivo projeto2_missao_orbital.c, compilado e testado;
- uma captura de tela com uma execução completa;
- nomes e RGMs dos integrantes em comentário no início do código;
- envio pelo link do Blackboard, dentro do prazo indicado pelo professor.

11. Regras importantes

- A atividade poderá ser realizada individualmente ou em grupos de até 3 alunos.
- O desenvolvimento acontecerá durante a aula e todos os integrantes deverão participar.
- O código deverá utilizar **`for, while, do-while`** e estruturas condicionais.
- Cada integrante deverá compreender o programa e conseguir explicar sua parte.
- Utilize nomes de variáveis claros, indentação correta e mensagens que orientem o usuário.
- Não será necessário utilizar vetores, matrizes, funções próprias ou manipulação de arquivos.
- A IA poderá apoiar o trabalho, mas o grupo deverá revisar, testar e compreender todo o código entregue. Além disso, o grupo deverá declarar o uso de IA, especificando em que parte e para qual finalidade foi utilizada.

12. Critérios de avaliação

Critério	Descrição
Funcionamento	O programa compila, executa, calcula corretamente e permite avaliar outro cadete.
Laços de repetição	Uso correto de for, while e do-while nas finalidades indicadas.
Condicionais	Validação, classificação e mensagem de pontuação máxima corretas.
Organização	Variáveis claras, indentação e saída legível.
Participação e explicação	O grupo trabalhou durante a aula e demonstrou compreensão da solução.
TOTAL	

13. Divisão sugerida para grupos

Integrante	Responsabilidade principal
1	Estrutura do programa, entradas e laço for.
2	Validação com while e cálculos de total e média.
3	Classificação, repetição com do-while e testes.

14. Desafios extras opcionais

- contar quantas etapas tiveram pontuação igual ou superior a 80;
- exibir a maior e a menor pontuação sem utilizar vetores;
- validar também a resposta 1-Sim ou 0-Não;
- personalizar o relatório com nome da nave e código da missão.

CHECKLIST FINAL Compilou? Validou 0 a 100? Repetiu três etapas? Classificou corretamente? Testou outro cadete?

Preparem a tripulação, testem cada etapa e boa missão!

Créditos: Projeto original proposto pelo Professor Vinícius Alves e adaptado para utilização em aula